

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **DATABASE MANAGEMENT SYSTEMS**

Subject Code: **CS T53**

UNIT I

Introduction: History – Purpose – View of Data – Database languages – Data Models – Data Storage and Querying – Transaction management – Database Architecture – Two tier – Three tier – Database users and Authorization.

Relational Algebra – Structure- keys – schema diagrams – Relational operations – Formal Relational Query Languages – Relational Algebra – Tuple Relational calculus – Domain Relational Calculus.

SQL Overview – Data Definition – basic structure – basic operations – Set Operations – Null Values – Aggregate Functions – Nested Subqueries – Modifications of the Databases – Join Expression – Integrity Constraints – Views – Authorization – Functions – Procedures – Triggers – Recursive Queries.

Unit I- Two Marks

1. What is DBMS? (NOV 2010) (DEC2018)

DBMS is a collection of interrelated data and a set of programs to access that data. The goal is to provide the environment that is both convenient and efficient to use in retrieving and storing data base information.

2. What are the advantages of DBMS? (APRIL 2014)

- Centralized Control
- Data Independence allows dynamic change and growth potential.
- Data Duplication is eliminated with control Redundancy.
- Data Quality is enhanced.
- Controlling redundancy
- Restricting unauthorized access
- Providing multiple user interfaces
- Enforcing integrity constraints.
- Providing back up and recovery

3. What are the Disadvantages of DBMS?

- Cost of software, hardware and migration is high.
- Complexity of Backup
- Problem associated with centralization.

4. List out the applications of DBMS.

- Banking

- Airlines
- Universities
- Credit card transactions
- Tele communication
- Finance
- Sales
- Manufacturing
- Human resources

5. What are the disadvantages of File Systems?

- Data Redundancy and Inconsistency.
- Difficulty in accessing data.
- Data Isolation
- Integrity problems
- Security Problems
- Concurrent access anomalies

6. Give the levels of data abstraction?

- a. Physical level
- b. Logical level
- c. View level

7. Define the terms

- a. Physical schema
- b. Logical schema.

Physical schema: The physical schema describes the database design at the physical level, which is the lowest level of abstraction describing how the data are actually stored.

Logical schema: The logical schema describes the database design at the logical level, which describes what data are stored in the database and what relationship exists among the data.

8. What is conceptual schema?

The schemas at the view level are called subschemas that describe different views of the database.

9. Define data model? (APRIL 2011)(NOV 2017)(DEC 2018)

A data model is a collection of conceptual tools for describing data, data relationships, data semantics and consistency constraints.

10.What is storage manager?

A storage manager is a program module that provides the interface between the low level data stored in a database and the application programs and queries submitted to the system.

11.What are the components of storage manager?

The storage manager components include

- a) Authorization and integrity manager
- b) Transaction manager
- c) File manager
- d) Buffer manager

12.What is the purpose of storage manager?

The storage manager is responsible for the following

- a) Interaction with the file manager
- b) Translation of DML commands in to low level file system commands
- c) Storing, retrieving and updating data in the database

13.List the data structures implemented by the storage manager

The storage manager implements the following data structure

- a) Data files
- b) Data dictionary
- c) Indices

14.What is a data dictionary?

A data dictionary is a data structure which stores meta data about the structure of the database ie. the schema of the database.

15.Explain Referential Integrity. (NOV 2017)

Referential Integrity means relationship between tables. Foreign keys are used. Foreign key is the column whose values are derived from the Primary key of the same or some other table.

Format for creating a foreign key is given below.

Syntax:

```
create table <table name>(columnname data type (size) constraint
constraint_name references parent table name);
```

16. Define Instances and schemas.

Instances:

The collection of information stored in the database at a particular moment is called the instance of the database.

Schemas:

The overall design of the database is called the schema of the database.

17. Define and explain the two types of Data Independence.

Two types of data independence are

Physical data independence:

The ability to modify the physical schema without causing application programs to be rewritten in the higher levels. Modifications in physical level occur occasionally whenever there is a need to improve performance.

Logical data independence:

The ability to modify the logical schema without causing application programs to be rewritten in the higher level (external level). Modifications in physical level occur frequently more than that in physical level, whenever there is an alteration in the logical structure of the database.

18. Define transaction.

A transaction is a collection of operations that performs a single logical function in a database application. Each transaction is a unit of both atomicity and consistency. Properties of transaction are atomicity, consistency, isolation, and durability.

19. Define the type types of DML.

Two types:

Procedural DML:

It requires a user to specify what data are needed and how to get those data.

Non-Procedural DML:

It requires a user to specify what data are needed without specifying how to get those data.

20. List out the functions of DBA.

- Schema definition
- Storage structure and access-method definition
- Schema and physical modification
- Granting of authorization for data access
- Integrity constraint specification

21. What is the need for DBA?

The need of DBA is to have central control of both the data and the programs that access those data. The person who has such central control over the system is called the DataBase Administrator. [

22. Define weak and strong entity sets? [NOV 2015]

Weak entity set

Entity set that do not have key attribute of their own are called weak

entity sets.

Strong entity set

Entity set that has a primary key is termed a strong entity set.

23.What does the cardinality ratio specify?

Mapping cardinalities or cardinality ratios express the number of entities to which another entity can be associated. Mapping cardinalities must be one of the following:

- One to one
- One to many
- Many to one
- Many to many

24.Explain the two types of participation constraint.

Total

The participation of an entity set E in a relationship set R is said to be total if every entity in E participates in at least one relationship in R.

Partial

if only some entities in E participate in relationships in R, the participation of entity set E in relationship R is said to be partial.

25.Explain DML pre-compiler.

DML precompiler converts DML statements embedded in an application program to normal procedure calls in the host language. The precompiler must interact with the DML compiler to generate the appropriate code.

26.Define file manager and buffer manager.

File manager:

File manager manages the allocation of space on disk storage and the data structures used to represent information stored on disk.

Buffer manager:

Buffer manager is responsible for fetching data from the disk storage into the main memory, and deciding what data to cache in memory.

27.Define Data Dictionary.

DDL statements are compiled into a set of tables that is stored in a special file called data dictionary. Data dictionary contains the meta-data, which in turn is data about the data.

28.What is a query language? [APRIL 2012, NOV 2014]

The query language is the language in which a user request information from the database. These languages are usually on a level higher than that of a standard programming language.

29. What in procedural and non procedural languages?

In procedural languages, the user instructs the system to perform a sequence of operation on the database to compute the desired result. In **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.

30. What in Cartesian –product operation?

The Cartesian –product operation denoted by a cross($\times$), allows us to combine information from any two relations. We write the Cartesian product of relations r_1 and r_2 as $r_1 \times r_2$.

31. Define domain?

For each attribute, there is a set of permitted values called the domain. For the attribute branch name, for example, the domain is the set of all branches name.

32. Define relation?

Relation to be a subset of a Cartesian product of a list of domains.

33. Define tuples?

A tuple variable is a variable that stands for a tuple, in other words a tuple variable is a variable whose domain is the set of all tuples.

34. What is the relation schema?

The concept of a relation schema corresponds to the programming –language notion of type definition. We use Account –schema to denote the relation schema for relation account. *Account-schema = (account-number, branch-name, balance)*. We denote the fact that account is a relation on Account –schema by *Account (Account-schema)*.

35. What is database schema?

Database schema in the logical design of the database.

36. What is database instance?

Database instance is a snapshot of the data in the database at a given instant in time.

37. What is relation instance?

The concept of a relation instance corresponds to the programming language notion of a value of a variable.

38. What is relational algebra? (NOV 2012)

The relational algebra is a procedural query language .It consists of set of operations that take one or two relations as input and produce a new relation as their result. The fundamental operation in the relational algebra is select, project, union, set difference, Cartesian product and rename.

39.What is select operation?

The select operation selects tuples that satisfy a given predicate .we use the lower case Greek letter sigma (σ).to denote selection. The predicate appears as a subscript to σ .The argument relation in parentheses after the σ

40.Describe extended relational operation? (DEC2018)

The basic relation –algebra operation have been extended in several ways. The simple extension is to allow arithmetic operations as part of projection. An important extension is to allow aggregate operations such as computing the sum of the elements of a set or their average.

41.What is meant by aggregate function? (APRIL 2012)

Aggregate function takes a collection of values and return a single value as a result.

42.What is meant by multiset?

The collections on which aggregate functions operates can have multiple Occurrences of value; the order in which the value appear in not relevant .such collections are called as multisets.

43.What is outer join?

Outer join operation is an extension of the join operation to deal with missing information. Suppose that we have the relations with the following schemas ,which contains data on full-time table name.

44.What are the types of outer join?

- left outer join
- right outer join
- full outer join

45.What is meant by null values?

The special values are called as Null values indicates “value unknown or non-existent”, any arithmetic operation such as (+,-,*, /) involving null values must return a null result.

46.In SQL operation how the different relational operations deal with null values?

- select
- join

- projection
- union, intersection, difference
- generalized projection
- aggregate
- outer join

47. Define modification of database?

It contains following operations like

- deletion
- insertion
- update

48. Define view? (NOV 2010, NOV 2015)

We define a view by using the **create view statement**. To define a view, we must give the view a name, and must state the query that computes the view. The form of **create view** statement is Create **view** *v* as <query expression>.

49. What is meant by tuple relation calculus?

The tuple relation calculus, by contrast, is a non-procedural query language. It describes the desired information without giving the specific procedure for obtaining that information. A query in the tuple relational calculus is expressed as $\{t | P(t)\}$.

50. What is meant by domain relation calculus? (APRIL 2012)

A second form of relational calculus are called domain relational calculus.

51. What are the parts of SQL languages?

There are several parts in SQL language

- Data-definition language (DDL)
- Interactive data manipulation language (DML)
- View definition
- Transaction control
- Embedded SQL and dynamic SQL
- Integrity
- Authorization

52. Explain the basic structure SQL?

The basic structure of an SQL expression consists of three clauses;

Select from where

53. Define select clause?

The **select** clause corresponds to the projection operation of the relational algebra. It is used to list the attribute desired in the result of query.

54. Define from clause?

The **from** clause corresponds to Cartesian –product operation of the relational algebra. It lists the relations, to be scanned in the evaluation of the expression.

55. Define where clause?

The where clause corresponds to selection predicate the relational algebra.

56. Write query for rename operation?

The query is *old- name as new-name*. eg:

select *customer- name ,borrower .loan- number ,amount*

from *borrower, Loan*

where *borrower .loan -number =loan. Loan- number.*

The result of this query is a relation with following attributes:

Customer –name ,loan-number, amount.

57. Define tuple variable?

The **as** clause is particularly useful in defining the notion of tuple variables, as is done in the tuple relational calculus. A tuple variable in SQL must be associated with a particular relation. Tuple variable are defined in the **from** clause by way of the **as** clause.

58. Define ordering the display of tuples?

The order by clause causes the tuples in the result of query to appear in sorted order . By default, the **order by** clause list item in ascending order to specify the sorted order, we may specify **desc** for descending order asc for ascending order.

59. What are the set operations available in SQL?

The SQL operations are

- union operation($\cup$)
- intersection operation($\cap$)
- except operation($-$)

60. What is union operation?

To find all the customer having a loan, and account , or both at bank, we write query as follow(**Select** *customer- name from depositor*)

Union

(**select** *customer name from borrower*)

The union operation automatically eliminates duplicates. If we want to retain all duplicates we must write **union all** in the place of **union**.

61. Describe intersection operation?

To find all the customer having both a loan, and account at a bank.

(**Select distinct** *customer- name from depositor*)

intersection

(select distinct customer name from borrower)

The **intersection** operation automatically eliminates duplicates . If we want to retain all duplicates we must write **intersection all** in the place of **intersection**.

62. Define aggregate function?

Aggregate functions are the functions **that** take a collection (a set or multiset) of values as input and return a single value.

63. What are the 5 built-in aggregate functions in SQL? [APRIL 2011]

- Average : **avg**
- Minimum : **min**
- Maximum : **max**
- Total : **sum**
- Count : **count**

64. Define group by clause?

In SQL using **group by** clauses, the attribute or attributes given in the **group by** clauses are used to form groups. Tuple with same values on all attributes in **group by** clause are placed in one group .we write query as: **Select branch name, avg (balance) from account group by branch name**

65. Define nested subqueries?

SQL provides a mechanism for nesting subqueries. A subquery is a select- from-where expression that is nested within another query.

66. Define with clause?

The with clause provides a way of defining a temporary view whose definition is available only to the query in which the **with** clause occurs.

67. What is meant by Transaction?

A transaction consists of a sequence of query and / or update statements. The SQL standard specifies that the transaction begins implicitly when an SQL statement is executed.

68. What are the types of transaction available in SQL?

Commit work: commits the current transaction; that is, it makes the updates performed by the transaction become permanent in the database . After the transaction is committed , a new transaction is automatically started.

Roll back work : It causes the current transaction to be rolled back ; that is , it undoes all the updates performed by SQL statements in the transaction

69. Domain types in SQL?

The SQL standard supports a variety of built-in domain types including

- Char (n)
- Varchar (n)
- Int
- Smallint
- numeric(p,d)
- real, double precision
- float(n)
- date
- time
- timestamp-(combination of date and time)

70.What is meant by check constraint?

Check (P): The check clause specify the predicate P that must be satisfied by the every tuple in the relation **Create** table account

```
(account-name char(10),
branch-name char(10),
balance integer,
primary key(account-number),
check (balance>=0))
```

71.Define ODBC?

The open database connectivity (ODBC) standard defines a way for a application program to communicate with a database server. ODBC defines an application program interface (**API**) that application can use to open a connection with database send queries and update and get back results.

72.What is meant JDBC?

JDBC standard defines an API the java program can use to connect to database server. JDBC-java database connectivity

73.Describe Query- by- Example(QBE)?

The QBE is the name of both a data manipulation language and an early database system that includes this language .

74.What is meant by condition box?

The condition box that allows the expression of general constrain over any of the domain variables. QBE allows logical expression to appear in a condition box . the logical operation are the words **AND** and **OR**, or the symbols "&"and"|".

75.What is meant by datalog?

The datalog is a non procedural query language based on the logic-programming prolog. As in the relation calculus a user describes the information decide without giving a specific procedure for obtaining that information. The syntax of datalog resembles that a prolog.

76.Describe user inter face and tools?

Although many people interact with database, few people use query language to directly interact with the database system. Most of the people interact with the database system through one of this following means:

- * Forms and GUI
- * Report generator
- Data analysis tools

77.Define forms and GUI?

Allows user to enter the values that complete predefined queries. The GUI (Graphical User Interface)provides an easy to use way to interact with the database System.

78.What is relational model? [April 2015]

The relational model used the basic concept of a relation or table. The columns or fields in the table identify the attributes such as name, age, and so. A tuple or row contains all the data of a single instance of the table such as a person named Doug. In the relational model, every tuple must have a unique identification or key based on the data.

79.Name any three RDBMS packages [April 2015]

Oracle (48.8%), IBM (20.2%), Microsoft (17.0%), SAP including Sybase (4.6%), and Teradata (3.7%).

80.What is integrity constraint?

Integrity constraint ensure that changes made to the database by authorized users do not result in a loss of data consistency.

81.What is domain constraint?

Domain constraint are the most elementary form of integrity constraint they are tested easily by the system whenever a new data item is entered into database

82.What is referential integrity?

To ensure a a value that appears in one relation for a given set of attributes also appears for certain set of attributes in another relation.

83.What is database modification?

- insert

- delete
- update

84.What is referential integrity in SQL?

Foreign key can be specified as a part of the sql 'create table' statement by using the foreign key clause. A foreign key references the primary key attributes of the referenced table. Example:

Create table stud (rno number (2),status varchar(5), foreign key(no) references student (regno));

85.What is assertions?

An assertion is a predicate expressing a condition that we wish the database always to satisfy.

An assertion in sol takes the form,

Query: Create assertion<assertion-name>check<predicate>

86.What is a trigger?

A trigger is a statement that the system executes automatically as a side effect of a modification to the database. There are two requirements to design a trigger mechanisms

- specify when a trigger is to executed. This is broken up into an event that causes the trigger to be checked and a condition that must be satisfied for trigger execution to proceed.
- specify the actions to be taken when the trigger executes.

87.Why triggers are needed? (DEC 2018)

Trigger are useful mechanisms for altering humans or for staring certain task automatically when certain conditions are met.

88.When not to use to trigger?

Triggers should not be used, when an insert trigger on a relation has an action that causes another insert action to be stimulated.

- Long chain of triggers should not be used, because it considers them as error.

89.What is security?

Data stored in the database need protection from unauthorized access and malicious destruction.

90.What is security violation?

- Unauthorized reading of data.
- Unauthorized modification of data
- Unauthorized destruction of data

91.What are levels of security?

- Database system

- Operating system
- Network
- Physical
- Human

92.What are the several form of authorization? (APRIL 2011)

- Read authorization-allows reading, but not modification of data
- Insert authorization- allows insertion of new data butr not modification of existing data
- Update authorization- allows modification but not deletion of data.
- Delete authorization-allows deletion
- Index authorization-allows the creation of new relations
- Resource authorization- allows the creation of new relations
- Alteration authorization- allows the addition or deletion of attributes in a relation
- Drop authorization-allows the deletion of relations

93.What is authorization graph?

The passing of authorization from one user to another can be represented by an authorization graph.

94.What is notion of roles?

Roles include teller, branch manager, system administrator. The notion of roles capture the schema. The set of roles is created in the database. Authorization can be granted to roles. Each database user is granted a set of roles.

95.What is an audit trail? (APRIL 2011)

An audit trail is a log of changes(insert, delete, update) to the databse along with information such as which user performed the change and when the change was performed.

96.What is encryption ?

Encryption forms the basis of good schemes for authenticating users to a database. The various provisions that a database may make for authorization may still not provide sufficient protection for high sensitive data.. in such cases data may be stored in encrypted form.

97.What are the encryption techniques?

A good encryption technique has the following techniques

- It is relatively simple for authorization users to encrypt and decrypt data.
- It depends not on the secrecy of the algorithm, but rather on a parameter of the algorithm called the encryption key.
- Its encryption key is extremely difficult for an intruder to determine.

98.What is authentication?

It refers to the task of verifying the identity of the person./software connecting to the database.

99. What is authorization?

The ultimate aim of authorization is to give authority to the database administrator. The database administrator may authorize new users, restructure the database using read, insert, update, delete, index, resource, alter, drop authorization.

100. What are the forms of assertion?

- Domain constraint
- Referential integrity constraint

101. What are the parts of SQL language?

The SQL language has several parts:

- data – definition language
- Data manipulation language
- View definition
- Transaction control
- Embedded SQL
- Integrity
- Authorization

102. What are the categories of SQL command?

SQL commands are divided in to the following categories:

1. data - definition language
2. data manipulation language
3. Data Query language
4. data control language
5. data administration statements
6. transaction control statements

103. What are the three classes of SQL expression?

SQL expression consists of three clauses:

Select

From

where

104. Give the general form of SQL query?

Select A , A , An

1 2

Fro m R , R , R

Where P

105. What is the use of rename operation?

Rename operation is used to rename both relations and attributes.

It uses the as clause, taking the form:

Old-name as new-name

106. Define tuple variable?

Tuple variables are used for comparing two tuples in the same relation. The tuple variables are defined in the from clause by way of the as clause.

107. List the string operations supported by SQL?

- 1) Pattern matching Operation
- 2) Concatenation
- 3) Extracting character strings
- 4) Converting between uppercase and lower case letters.

108. List the set operations of SQL?

- 1) Union
- 2) Intersect operation
- 3) The except operation

109. What is the use of Union and intersection operation?

Union : The result of this operation includes all tuples that are either in r1 or in r2 or in both r1 and r2. Duplicate tuples are automatically eliminated.

Intersection: The result of this relation includes all tuples that are in both r1 and r2.

110. What are aggregate functions? And list the aggregate functions supported by SQL?

Aggregate functions are functions that take a collection of values as input and return a single value. Aggregate functions supported by SQL are

Average: avg

Minimum: min

Maximum: max

Total: sum

111. What is the use of group by clause?

Group by clause is used to apply aggregate functions to a set of tuples. The attributes given in the group by clause are used to form groups. Tuples with the same value on all attributes in the group by clause are placed in one group.

112. What is the use of sub queries?

A sub query is a select-from-where expression that is nested with in another query. A common use of sub queries is to perform tests for set membership, make set comparisons, and determine set cardinality.

113. What is view in SQL? How is it defined? NOV 2014

Any relation that is not part of the logical model, but is made visible to a user as a virtual relation is called a view. We define view in SQL by using the create view command. The form of the create view command is

Create view v as <query expression>

114. What is the use of with clause in SQL?

The with clause provides a way of defining a temporary view whose definition is available only to the query in which the with clause occurs.

115. List the table modification commands in SQL?

Deletion
Insertion
Updates
Update of a view

116. List out the statements associated with a database transaction?

Commit work
Rollback work

117. What is transaction?

Transaction is a unit of program execution that accesses and possibly updated various data items.

118. List the SQL domain Types?

SQL supports the following domain types.

1) Char(n) 2) varchar(n) 3) int 4) numeric(p,d)
5) float(n) 6) date.

119. What is the use of integrity constraints?

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus integrity constraints guard against accidental damage to the database.

120. Mention the 2 forms of integrity constraints in ER model?

Key declarations
Form of a relationship

121. What is trigger? NOV 2010

Triggers are statements that are executed automatically by the system as the side

effect of a modification to the database.

122. What are domain constraints?

A domain is a set of values that may be assigned to an attribute .all values that appear in a column of a relation must be taken from the same domain.

123. What are referential integrity constraints? NOV 2010

A value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.

124. What is assertion? Mention the forms available.

An assertion is a predicate expressing a condition that we wish the database always to satisfy.

Domain integrity constraints.

Referential integrity constraints

125. Give the syntax of assertion?

Create assertion <assertion name> check <predicate>

126. What is the need for triggers?

Triggers are useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met.

127. List the requirements needed to design a trigger.

The requirements are

Specifying when a trigger is to be executed.

Specify the actions to be taken when the trigger executes.

128. Give the forms of triggers?

The triggering event can be insert or delete.

For updated the trigger can specify columns.

The referencing old row as clause

The referencing new row as clause

The triggers can be initiated before the event or after the event.

129. What does database security refer to?

Database security refers to the protection from unauthorized access and malicious destruction or alteration.

130. List some security violations (or) name any forms of malicious access.

Unauthorized reading of data

Unauthorized modification of data

Unauthorized destruction of data.

131. List the types of authorization.

Read authorization

Write authorization
Update authorization
Drop authorization

132. What is authorization graph?

Passing of authorization from one user to another can be represented by an authorization graph.

133. List out various user authorization to modify the database schema.

Index authorization
Resource authorization
Alteration authorization
Drop authorization

134. What are audit trails?

An audit trail is a log of all changes to the database along with information such as which user performed the change and when the change was performed

135. Mention the various levels in security measures.

Database system
Operating system
Network
Physical
human

136. Name the various privileges in SQL?

Delete
Select
Insert
update

137. Mention the various user privileges.

All privileges directly granted to the user or role.
All privileges granted to roles that have been granted to the user or role.

138. Give the limitations of SQL authorization.

The code for checking authorization becomes intermixed with the rest of the application code.

Implementing authorization through application code rather than specifying it declaratively in SQL makes it hard to ensure the absence of loopholes.

139. Give some encryption techniques?

DES

AES

Public key encryption

140. What does authentication refer?

Authentication refers to the task of verifying the identity of a person.

141. List some authentication techniques.

Challenge response scheme

Digital signatures

Nonrepudiation

142. List four difference between File Processing and DBMS. (APRIL 2018)

- Redundancy is control in DBMS, but not in file system
- Unauthorized access is restricted in DBMS but not in file system.
- DBMS provide back up and recovery. When data is lost in file system then it not recover.
- DBMS provide multiple user interfaces. Data is isolated in file system,

143. Explain Set-difference operation. (APRIL 2018)

Set Difference Operation (-)

To find tuples that is in one relation but is not in another. The two condition of union operation also apply for set difference. Syntax: Relation1 - Relation 2 Example: find all customers of the bank who have an account but not a loan.

$\Pi_{\text{customer-name}}(\text{depositor}) - \Pi_{\text{customer-name}}(\text{borrower})$

The result relation for this query Customer Table after Set Difference Customer name

Johnson

Lindsay

Turner

11 MARK QUESTION AND ANSWERS

1. Explain in detail about Database management systems(6 Marks)

Database Management System (DBMS) is a Collection of interrelated data s Set of programs to access the data s DBMS contains information about a particular enterprise s DBMS provides an environment that is both convenient and efficient to use.

- Database Applications:
- Banking: all transactions
- Airlines: reservations
- Universities: registration, grades
- Sales: customers, products, purchases
- Manufacturing: production, inventory, orders, supply chain
- Human resources: employee records, salaries, tax deductions

Database applications were built on top of file systems. Drawbacks of using file systems to store data:

- Data redundancy and inconsistency Multiple file formats, duplication of information in different files.
- Difficulty in accessing data Need to write a new program to carry out each new task
- Data isolation — multiple files and formats
- Integrity problems Integrity constraints (e.g. account balance > 0) become part of program code Hard to add new constraints or change existing ones Database System Concepts
- Atomicity of updates Failures may leave database in an inconsistent state with partial updates carried out E.g. transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users Concurrent accessed needed for performance Uncontrolled concurrent accesses can lead to inconsistencies – E.g. two people reading a balance and updating it at the same time
- Security problems s Database systems offer solutions to all the above problems

2. Explain in detail about view of data and levels of Abstraction(11 Marks)

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.

Data Abstraction:

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

Levels of Abstractions:

- **Physical level** describes how a record (e.g., customer) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data. The Logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.
- **View level.** The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database. Figure 1.1 shows the relationship among the three levels of abstraction.

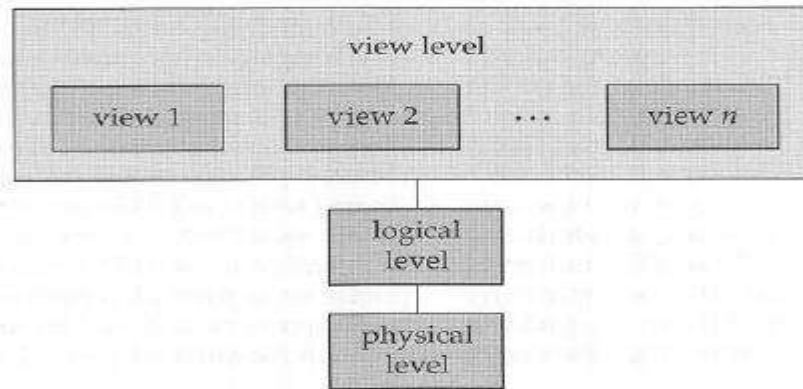


Fig 1.1 Three levels of abstraction.

- For example, in a Pascal-like language/ we may declare a record as follows:

type customer = record

name: string;

street: string;

city: string;

Street: string;

end;

- This code defines a new record type called customer with four fields. Each field has a name and a type associated with it. A banking enterprise may have several such record types, including
 - **Account with fields account_number and balance**
 - **Employee with fields employee-name and salary**
- At the physical level, a customer, account, or employee record can be described as a block of consecutive storage locations (for example, words or bytes). The compiler hides this level of detail from programmers. Similarly, the database system hides many of the lowest level storage details from database programmers. Database administrators, on the other hand, may be aware of certain details of the physical organization of the data.
- At the logical level, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well. Programmers using a programming language work at this level of abstraction. Similarly, database administrators usually work at this level of abstraction.

- Finally, at the view level, computer users see a set of application programs that hide details of the data types. Similarly, at the view level several views of the database are defined, and database users see these views. In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database. For example, tellers in a bank see only that part of the database that has information on customer accounts; they cannot access information about salaries of employees.

3. Explain in detail about Instances & Schemas(5 Marks)

Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. The overall design of the database is called the database schema. Schemas are changed infrequently, if at all.

The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an instance of a database schema.

Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas that describe different views of the database.

Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit physical data independence if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

4. Explain in detail about Data Models(6 Marks) (DEC2018)

The data model is defined as a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. A data model provides a way to describe the design of a database at the physical, Logical, and view level. There are a number of different data models. The data models can be classified in four different categories:

1. Relational Model
2. The Entity-Relationship Model
3. Object-Based Data Model.
4. Semi Structured Data Model

Relational Model: The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model.

Entity-Relationship Model: The entity-relationship (E-R) data model is based on a perception of a real world that consists of a collection of basic objects called entities and of relationships among these objects. An entity is a "thing" or "object" in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design.

Object-Based Data Model: The object-oriented data model is another data model that has seen increasing attention. The object-oriented model can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity. The object-relational data model combines features of the object-oriented data model and relational data model.

Semi structured Data Model: The semi structured data model permits the specification of data where individual data items of the same type may have different sets of attributes. This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes. The Extensible Markup Language (XML) is widely used to represent semi structured data.

5. Explain in detail about Database Languages and its types(11 Marks)

A database system provides a data-definition language to specify the database schema and a data-manipulation language to express database queries and updates. In practice, the data-definition and data-manipulation languages are not two separate language instead they simply form parts of a single database language, such as the widely used SQL language.

Data Manipulation Language: A data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:

- Retrieval of information stored in the database
- Insertion of new information into the database
- Deletion of information from the database
- Modification of information stored in the database

There are basically two types:

- Procedural DMLs require a user to specify what data are needed and how to get those data.
- Declarative DMLs (also referred to as nonprocedural DMLs) require a user to specify what data are needed without specifying how to get those data.

Declarative DMLs are usually easier to learn and use than are procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data. A query is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a query language. Although technically incorrect, it is common practice to use the terms query language and data manipulation language synonymously.

Data Definition Language: We specify a database schema by a set of definitions expressed by a special language called a data-definition language (DDL). The DDL is also used to specify additional properties of the data. We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a data storage and definition language. These statements define the implementation details of the database schemas, which are usually hidden from the users.

The data values stored in the database must satisfy certain consistency constraints. For example, suppose the balance on an account should not fall below RS. 100. The DDL provides facilities to specify such constraints. The database systems check these constraints every time the database is updated. In general, a constraint can be an arbitrary predicate pertaining to

the database. However, arbitrary predicates may be costly to test. Thus, database systems concentrate on integrity constraints that can be tested with minimal overhead.

Domain Constraints: A domain of possible values must be associated with every attribute (for example, integer types, character types, date/time types). Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take. Domain constraints are the most elementary form of integrity constraint. They are tested easily by the system whenever a new data item is entered into the database.

Referential Integrity: There are cases where we wish to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation (referential integrity). Database modifications can cause violations of referential integrity. When a referential integrity constraint is violated, the normal procedure is to reject the action that caused the violation.

Assertions: An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. However, there are many constraints that we cannot express by using only these special forms. For example, "Every loan has at least one customer who maintains an account with a minimum balance of Rs.1000.00 must be expressed as an assertion. When an assertion is created, the system tests it for validity. If the assertion is valid, then any future modification to the database is allowed only if it does not cause that assertion to be violated.

Authorization: We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of authorization, the most common being, **read authorization**, which allows reading, but not modification, of data; **insert authorization**, which allows insertion of new data, but not modification of existing data; **update authorization**, which allows modification, but not deletion, of data and **delete authorization**, which allows deletion of data. We may assign the user all, none, or a combination of these types of authorization

The DDL, just like any other programming language, gets as input some instructions (statements) and generates some output. The output of the DDL is placed in the data dictionary, which contains metadata-that is, data about data. The data dictionary is considered to be a special type of table, which can only be accessed and updated by the database system itself (not a regular user). A database system consults the data dictionary before reading or modifying actual data.

6. **Describe the Structure of Relational Databases** (This Questions consists of 4 Parts where each having the weightage of asking separately in 6 marks, 11 marks and 5 Marks) **NOV 2014[DEC 2016] (APRIL 2018)**

A relational database consists of a collection of tables, each of which is assigned a unique name. A row in a table represents a relationship among a set of values. Informally, a table is an entity set, and a row is an entity. Since a table is a collection of such relationships, there is a close correspondence between the concept of table and the mathematical concept of relation, from which the relational data model takes its name. In what follows, we introduce the concept of relation.

1. **Basic Structure of Relational Database(11 Marks)**
2. **Database Schema & Schema Diagram of Relational Database(11 Marks)**
3. **Keys of Relational Database(6 Marks)**
4. **Query Languages(4 marks)**

(i).Basic Structure of Relational Database(11 Marks)

Consider the account table of Figure 1.2 shown below. It has three column headers: account_number, branch-name and balance.

<i>account_number</i>	<i>branch_name</i>	<i>balance</i>
A-101	Downtown	500
A-102	Perryridge	400
A-201	Brighton	900
A-215	Mianus	700
A-217	Brighton	750
A-222	Redwood	700
A-305	Round Hill	350

1.2 Account Relation

For each attribute, there is a set of permitted values, called the domain of that attribute. For the attribute branch-name for example, the domain is the set of all branch names. Let D_1 denote the set of all account numbers, D_2 the set of all branch names, and D_3 the set of all balances. Any row of account must consist of a 3-tuple (u_1, u_2, u_3) where ' u_i ' is an account number (that is, u_1 is in domain D_1), u_2 is a branch name (that is, u_2 is in domain D_2), and u_3 is a balance (that is, u_3 is in domain D_3). In general, account will contain only a subset of the set of all possible rows. Therefore, account is a subset of $D_1 \times D_2 \times D_3$.

(ii). Database Schema & Schema Diagram of Relational Database (11 Marks)
(APRIL 2018)

The database schema, which is the logical design of the database, and the database instance, which is a snapshot of the data in the database at a given instant in time. The concept of a relation corresponds to the programming-language notion of a variable. The concept of a relation schema corresponds to the programming-language notion of type definition.

It is convenient to give a name to a relation schema, just as we give names to type definitions in programming languages. We adopt the convention of using lowercase names for

relations, and names beginning with an uppercase letter for relation schemas. Following this notation, we use Account-schema to denote the relation schema for relation account. Thus,

“Account_schema = (account_number, branch_name, balance)”

In general, a relation schema consists of a list of attributes and their corresponding domains. The concept of a relation instance corresponds to the programming-language notion of a value of a variable. The value of a given variable may change with time; similarly the contents of a relation instance may change with time as the relation is updated. However, we often simply say "relation" when we actually mean "relation instance."

As an example of a relation instance consider the branch relation of Figure 1.4. as shown below

<i>branch_name</i>	<i>branch_city</i>	<i>assets</i>
Brighton	Brooklyn	7100000
Downtown	Brooklyn	9000000
Mianus	Horseneck	400000
North Town	Rye	3700000
Perryridge	Horseneck	1700000
Pownal	Bennington	300000
Redwood	Palo Alto	2100000
Round Hill	Horseneck	8000000

Figure1.4. Branch Relation

The schema for that relation is

“Branch_schema=(branch_name, branch_city, assets)”

Note that the attribute branch-name appears in both Branch_schema and Account _schema. This duplication is not a coincidence. Rather, using common attributes in relation schemas is one way of relating tuples of distinct relations. For example, suppose we wish to find the information about all of the accounts maintained in branches located in Brooklyn. We look first at the branch relation to find the names of all the branches located in Brooklyn. Then, for each such branch, we look in the account relation to find the information about the accounts maintained at that branch.

Let us continue our banking example. We need a relation to describe information about customers. The relation schema is

“Customer_schema = (customer_name, customer_street, customer_city)”

<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>
Adams	Spring	Pittsfield
Brooks	Senator	Brooklyn
Curry	North	Rye
Glenn	Sand Hill	Woodside
Green	Walnut	Stamford
Hayes	Main	Harrison
Johnson	Alma	Palo Alto
Jones	Main	Harrison
Lindsay	Park	Pittsfield
Smith	North	Rye
Turner	Putnam	Stamford
Williams	Nassau	Princeton

Figure1.5 Customer Relation

The above Figure 1.5 shows a sample relation customer (Customer_schema). Note that we have omitted the customer-id attribute that we used in previous diagrams, because now we want to have smaller relation schemas in our running example of a bank database.

We assume that the customer name uniquely identifies a customer-obviously this may not be true in the real world, but the assumption makes our examples much easier to read. In a real-world database, the customer-id (which could be a social-security number or an identifier generated by the bank) would serve to uniquely identify customers. We also need a relation to describe the association between customers and accounts. The relation schema to describe this association is

“Depositor_schema = (customer_name, account_number)”

Fig 1.6 shows a sample relation depositor (Depositor-schema).

<i>customer_name</i>	<i>account_number</i>
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Smith	A-215
Turner	A-305

Figure1.6 Depositor Relation

It would appear that, for our banking example, we could have just one relation schema, rather than several. That is, it may be easier for a user to think in terms of one relation schema, rather than in terms of several. Suppose that we used only one relation for our example, with schema.

**“(Branch_name, branch_city, assets, customer_name, customer_street
Customer_city, account_number, balance)”**

Observe that, if a customer has several accounts, we must list her address once for each account. That is, we must repeat certain information several times. This repetition is wasteful

and is avoided by the use of several relations, as in our example' In addition, if a branch has no accounts (a newly created branch, say, that has no customers yet), we cannot construct a complete tuple on the preceding single relation, because no data concerning customer and account are available yet. To represent incomplete tuples, we must use null values that signify that the value is unknown or does not exist. Thus, in our example, the values for **customer_name**, **customer_street**, and so on must be null. By using several relations, we can represent the branch information for a bank with no customers without using null values. We simply use a tuple on Branch_schema to represent the information about the branch, and create tuples on the other schemas only when the appropriate information becomes available.

Schema Diagram

A database schema, along with primary key and foreign key dependencies, can be depicted pictorially by schema diagrams. Figure 1.7 shows the schema diagram for our banking enterprise. Each relation appears as a box, with the attributes listed inside it and the relation name above it. If there are primary key attributes, a horizontal line crosses the box, with the primary key attributes listed above the line in gray. Foreign key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation. Many database systems provide design tools with a graphical user interface for creating schema diagrams.

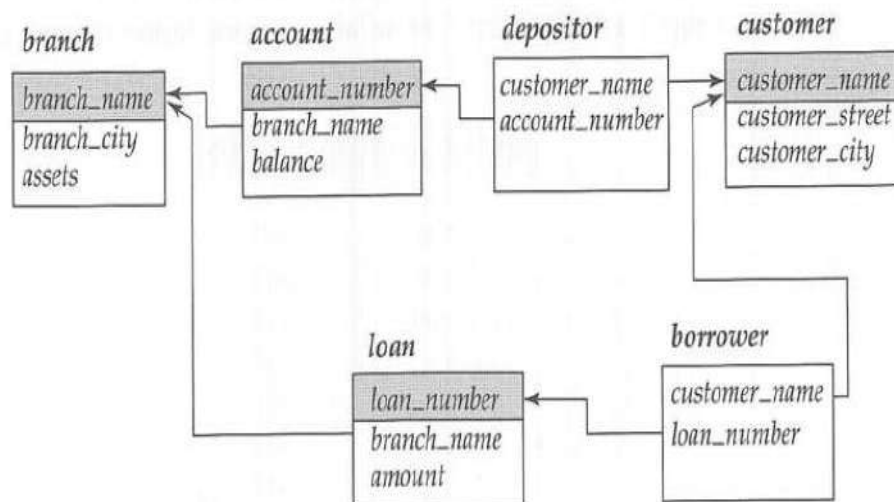


Figure1.6 Schema Diagram of a Banking Enterprise

(iii). Keys of Relational Database (6 Marks)

We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes. That is, the values of the attribute values of a tuple must be such that they can uniquely identify the tuple. In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes.

Keys: $K \subseteq R$

K is a super key of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$. By "possible r" we mean a relation r that could exist in the enterprise we are modeling.

Example: {customer_name, customer_street} and {customer_name} are both super keys of Customer, if no two customers can possibly have the same name. In real life, an attribute such as customer_id would be used instead of customer_name to uniquely identify customers, but we omit it to keep our examples small, and instead assume customer names are unique.

Types of keys:

1) K is a **candidate key** if K is minimal

Example: {customer_name} is a candidate key for Customer, since it is a super key and no subset of it is a super key.

2) **Primary key:** a candidate key chosen as the principal means of identifying tuples within a relation should choose an attribute whose value never, or very rarely, changes.

E.g. email address is unique, but may change.

3) **Foreign Keys(2 MARKS NOV 2012),APRIL 2014.**

A relation schema may have an attribute that corresponds to the primary key of another relation. The attribute is called a foreign key.

E.g. customer_name and account_number attributes of depositor are foreign keys to customer and account respectively.

Only values occurring in the primary key attribute of the referenced relation may occur in the foreign key attribute of the referencing relation.

(iv) Query Languages (4 marks)

A query language is a language in which a user requests information from the database. These languages are usually on a level higher than that of a standard programming Language. Query languages can be categorized as either procedural or nonprocedural. In a procedural language, the user instructs the system to perform a sequence of operations on the database to compute the desired result. In a nonprocedural language, the user describes the desired information without giving a specific procedure for obtaining that information.

Most commercial relational database systems offer a query language that includes elements of both the procedural and the nonprocedural approaches. There are a number of "pure" query languages: The relational algebra is procedural, whereas the tuple relational calculus and domain relational calculus are nonprocedural. These query languages are terse and formal, lacking the "syntactic sugar" of commercial languages, but they illustrate the fundamental techniques for extracting data from the database.

7. Describe Relational Algebra and its operations(11 Marks).NOV2014 ,NOV 2015

Relation Algebra(2 MARKS NOV 2012) (NOV 2017)

The relational algebra is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation as their result.

The operations can be divided into,

- **Basic operations:** Select, Project, Union, rename, set difference and Cartesian product
- **Additional operations:** Set intersections, natural join, division and assignment.
- **Extended operations:** Aggregate operations and outer join
-

The fundamental operations are :

1. Select
2. Project
3. Union
4. Set difference
5. Cartesian product and
6. Rename.

Example: Consider simple Relation r

A	B	C	D
A	a	1	7
A	b	5	7
B	b	12	3
B	b	23	10

$\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
A	a	1	7
B	b	23	10

SELECTION OPERATION

- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

- Where p is a formula in propositional calculus consisting of **terms** connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Each **term** is one of:
<Attribute> op <attribute> or <constant>
 - where op is one of: $=, \neq, >, \geq, <, \leq$

- **Select:** It selects tuples that satisfy a given predicate, To denote selection. (Sigma) is used.

Syntax

$$\sigma_{\text{condition}}(\text{table name})$$

Example

$\sigma_{\text{sal} > 1000}(\text{emp})$ ---- It selects tuples whose employee sal is > 1000

PROJECT OPERATION

- It selects attributes from the relation.
- Symbol for project

Syntax:

$$\Pi_{\text{<Attribute list>}}(\text{Table name})$$

Example:

$$\Pi_{\text{eid, sal}}(\text{Employee})$$

- Combining Select & Project Operation

$$\Pi_{\text{eid, sal}}(\sigma_{\text{Sal} > 1000}(\text{employee}))$$

- Selects tuples where sal > 1000 & from them only eid and salary attributes are selected.

UNION OPERATION

Consider a query to find the names of all bank customers who have either an account or a loan or both. Note that the customer relation does not contain the information, since a customer does not need to have either an account or a loan at the bank.

1. We know how to find the names of all customers with a loan in the bank:

$\Pi_{\text{customer-name}}(\text{borrower})$

2. We also know how to find the names of all customers with an account in the bank:

$\Pi_{\text{customer-name}}(\text{depositor})$

The binary operation union, denoted, as in set theory, by U. So the expression needed is $\Pi_{\text{customer-name}}(\text{borrower}) \cup \Pi_{\text{customer-name}}(\text{depositor})$. Therefore, for a union operation $r \cup s$ to be valid, we require that two conditions hold:

1. The relations r and s must be of the same arity. That is, they must have the same number of attributes.
2. The domains of the i th attribute of r and the i th attribute of s must be the same, for all i .

THE SET DIFFERENCE OPERATION:

1. The set-difference operation, denoted by $-$, allows us to find tuples that are in one relation but are not in another.
2. The expression $(r - s)$ produces a relation containing those tuples in r but not in s .
3. We can find all customers of the bank who have an account but not a loan by writing $\Pi_{\text{customer-name}} (\text{depositor}) - \Pi_{\text{customer-name}} (\text{borrower})$

THE CARTESIAN-PRODUCT OPERATION

1. The Cartesian-product operation, denoted by a cross ($\times$), allows us to combine information from any two relations.
2. We write the Cartesian product of relations r_1 and r_2 as $(r_1 \times r_2)$.
3. Suppose that we want to find the names of all customers who have a loan at the Perryridge branch. We need the information in both the loan relation and the borrower relation to do so.
4. If we write $\sigma_{\text{branch-name} = \text{"Perryridge"}} (\text{borrower} \times \text{loan})$ However, the customer-name column may contain customers
5. Finally, since we want only customer-name, we do a projection
 $\Pi_{\text{customer-name}} (\sigma_{\text{borrower}.\text{loan-number} = \text{loan}.\text{loan-number}} (\sigma_{\text{branch-name} = \text{"Perryridge"}} (\text{borrower} \times \text{loan})))$

THE RENAME OPERATION:

1. Unlike relations in the database, the results of relational-algebra expressions do not have a name that we can use to refer to them.
2. It is useful to be able to give them names; the rename operator, denoted by the lowercase Greek letter rho (ρ).

Consider the following example query:

Find the names of all customers who live on the same street and in the same city as Smith."

We can obtain Smith's street and city by writing

$\Pi_{\text{customer-street, customer-city}} (\sigma_{\text{customer-name} = \text{"Smith"}} (\text{customer}))$

However, in order to find other customers with this street and city, we must reference the customer relation a second time. In the following query, we use the rename operation on the preceding expression to give its result the name `smith-addr`, and to rename its attributes to `street` and `city`, instead of `customer-street` and `customer-city`:

$\Pi_{\text{customer.customer-name}}$

$(\sigma_{\text{customer.customer-street} = \text{smith-addr.street}} \rho$

$\text{Customer.customer-city} = \text{smith-addr.city})$

(customer × psmith-addr (street, city)

(Π(customer-street, customer-city (σ(customer-name = "Smith" (customer))))))

8. Explain in detail about SQL with its structure and example(This question was asked with some selective queries with example)(11 marks)

SQL

History

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:
 - SQL86
 - SQL89
 - SQL92
 - SQL: 1999 (language name became Y2K compliant!)
 - SQL:2003
- Commercial systems offer most, if not all, SQL92 features, plus varying feature sets from later standards and special proprietary features.
 - Not all examples here may work on your particular system.

SQL is nothing but a set of commands/statements and are categorized into following five groups viz., DQL: Data Query Language, DML: Data Manipulation Language, DDL: Data Definition Language, TCL: Transaction Control Language, DCL: Data Control Language.

DQL: SELECT

DML: DELETE, INSERT, UPDATE

DDL: CREATE, DROP, TRUNCATE, ALTER

TCL: COMMIT, ROLLBACK, SAVEPOINT

DCL: GRANT, REVOKE

Data Definition Language: Allows the specification of not only a set of relations but also information about each relation, including

- The schema for each relation.
- The domain of values associated with each attribute.
- Integrity constraints
- The set of indices to be maintained for each relations.
- Security and authorization information for each relation.
- The physical storage structure of each relation on disk.

Select

- 1) `SELECT *
FROM emp;`
- 2) `SELECT ename, salary
FROM emp;`
- 3) `SELECT ename, salary
FROM emp
WHERE salary > 2000;`
- 4) `SELECT ename, salary
FROM emp
WHERE salary > 2000
ORDER BY ename;`

The SELECT statement as the name says is used to extract data from Oracle Database. The syntax for the simplest SELECT statement is as follows.

```
SELECT column_name1, column_name2, ...  
FROM table_name1;
```

Example:

```
SELECT * FROM emp;
```

This command will display all the fields of the table emp and all of the records.

Example:

```
SELECT ename, sal FROM emp WHERE sal > 2000;
```

The result of this statement will be only two columns of emp table and only those records where salary is greater than 2000.

Example:

```
SELECT ename, salary  
FROM emp  
WHERE sal > 2000  
ORDER BY ename;
```

The output of this statement will be exactly the same as the one above except that the output will be sorted based on ename column.

SQL Operators

Comparison Operators	Arithmetic Operators
1. =	1. +
2. <	2. -
3. >	3. /
4. <=	4. *
5. >=	
6. ANY or SOME	
7. ALL	
Logical Operators	Other
1. NOT	• IN
2. AND	• BETWEEN
3. OR	• EXISTS
	• LIKE
	• IS NULL
	• IS NOT NULL

Figure - SQL Operators: Comparison, Arithmetic, Logical & Other.

Example:

```
SELECT ename, sal, sal + sal*5/100 "Next Year Sal"  
FROM emp;
```

Logical Operators and the operators in the Other category can be best understood by looking

at their respective real world examples.

Example:

```
SELECT sal FROM emp WHERE deptno = 30 AND sal > 2000;
```

The output of the query will be only one column i.e. sal and only those records will be displayed where department number is 30 and the salary is greater than 2000. So when you use AND operator it means both conditions need to satisfy for the record to appear in the output but in case of OR, either first condition needs to be true or the second one e.g.

```
SELECT sal FROM emp WHERE deptno = 30 OR sal > 2000;
```

Example:

```
SELECT * FROM emp WHERE job IN ('CLERK','ANALYST');
```

The output of the query will be all the columns of emp table but only those records where job column contains either "CLERK" or "ANALYST". You can also use IN operator with as follows.

```
SELECT *
```

```
FROM emp WHERE sal IN (SELECT sal FROM emp WHERE deptno = 30);
```

By having NOT before IN can completely invert the results like in the following example. Such types of queries fall under the category called "Sub-Queries" which we will discuss in the article ahead in this chapter. There is a special technique to interpret them.

```
SELECT *
```

```
FROM emp WHERE sal NOT IN (SELECT sal FROM emp WHERE deptno = 30);
```

Example:

```
SELECT * FROM emp WHERE sal BETWEEN 2000 AND 3000;
```

Only those records will be displayed where the salary is between 2000 and 3000 including both 2000 and 3000.

Example:

```
SELECT ename, deptno FROM dept WHERE EXISTS (SELECT * FROM emp WHERE  
dept.deptno = emp.deptno);
```

TRUE if a sub-query returns at least one row. In other words the output will be two columns from dept table, all the records only if the query after EXISTS results in at least one record.

Example:

```
SELECT sal FROM emp WHERE ename LIKE 'SM%';
```

The output will be only those salaries from emp (employee) table where ename (employee name) begins with "SM". Another variation of above query is as follows.

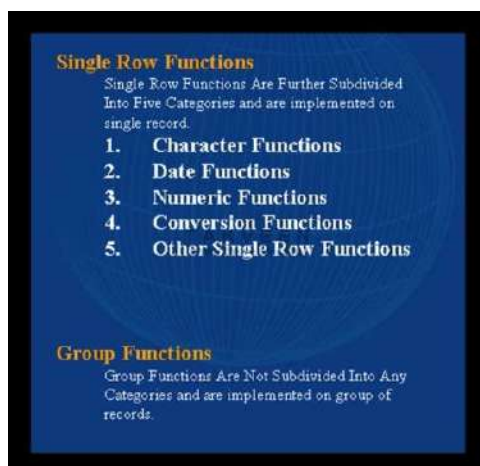
```
ename LIKE 'SMITH_'
```

The output will be only those records where ename begins with "SMITH" and there should not be more than one character after it.

Example:

```
SELECT ename, deptno FROM emp WHERE comm IS NULL;
```

The output will be ename and deptno but only those records where comm field has NULL value. NULL is a special value and just keep in mind that its not Zero. It can be visualized as empty field occupying zero byte

Functions**Single Row Functions:**

Single Row functions are further subdivided into five categories viz., Character, Data, Numeric, Conversion & other functions. First we will start with Character Functions or more precisely "Single Row Character Functions."

Character Functions

Following are the functions that fall under this category.

CHR, , LTRIM, INSTR, ASCII, RTRIM, INSTRB, CONCAT, TRIM, LENGTH, INITCAP, REPLACE, LENGTHB, LOWER, SOUNDex, UPPER, SUBSTR, LPAD, SUBSTRB, RPAD.

Example:

```
SELECT CHR(67)||CHR(65)||CHR(84) "Pet" FROM DUAL;
```

Output:

Pet

CAT

Example:

```
SELECT ASCII('Q') FROM DUAL;
```

Output:

ASCII('Q')

81

Example:

```
SELECT CONCAT(ename, ' is a good boy') "Result"
FROM emp
WHERE empno = 7900;
```

Output:

Result

JAMES is a good boy

Example:

SELECT INITCAP('the king') "Capitals" FROM DUAL;

Output:

Capitals

The King

Example:

SELECT LOWER('THE KING') "Lowercase" FROM DUAL;

Output:

Lowercase

the king

Similarly we can use the UPPER function.

Example:

SELECT LPAD('Page 1',15,'*+') "LPAD example" FROM DUAL;

Output:

LPAD example

++*+*+*Page 1

Similarly we can use RPAD.

Example:

SELECT LTRIM('121SPIDERMAN','12') "Result" FROM DUAL;

Output:

Result

SPIDERMAN

Similarly we can use RTRIM.

Example:

SELECT TRIM (0 FROM 001234567000) Result" FROM DUAL;

Output:

Result

1234567

SELECT TRIM (LEADING 0 FROM 001234567000) Result" FROM DUAL;

Output:

Result

1234567000

Similarly we can replace LEADING with TRAILING to omit trailing zeros in 001234567000 and the output will then be 001234567.

Example:

SELECT REPLACE('KING KONG','KI','HO') Result" FROM DUAL;

Output:

Changes

HONG KONG

Example:

SELECT ename FROM emp

WHERE SOUNDEX(ename) = SOUNDEX('SMYTHE');

Output:

ENAME

SMITH

Example:

SELECT SUBSTR('SPIDERMAN',7,3) "Result"

FROM DUAL;

Output:

Result

MAN

Similarly we can use SUBSTRB; for a single-byte database character set, SUBSTRB is equivalent to SUBSTR. Floating-point numbers passed as arguments to SUBSTRB are automatically converted to integers. Assume a double-byte database character set:

SELECT SUBSTRB(' SPIDERMAN',7,4.3) "Result" FROM DUAL;

Output:

Result

DE

Example:

SELECT INSTR('CORPORATE FLOOR','OR', 3, 2) "Instring" FROM DUAL;

Output:

Result

14

Similarly we can use INSTRB; for a single-byte database character set, INSTRB is equivalent to INSTR. Lets suppose a double-byte database character set.

SELECT INSTRB('CORPORATE FLOOR','OR',5,2) "Result" FROM DUAL;

Output:

Result

27

Example:

SELECT LENGTH('SPIDERMAN') "Result"FROM DUAL;

Output:

Result

9

Similarly we can use LENGTHB; for a single-byte database character set, LENGTHB is equivalent to LENGTH. Lets suppose a double-byte database character set.

SELECT LENGTHB ('SPIDERMAN') "Result"FROM DUAL;

Output:

Result

14

DATE FUNCTIONS:

Following functions fall under this category.

ADD_MONTHS

MONTHS_BETWEEN

LAST_DAY

ROUND

SYSDATE

TRUNC

Example:

```
SELECT ADD_MONTHS(hiredate,1)FROM emp WHERE ename = 'SMITH';
```

Example:

```
SELECT SYSDATE, LAST_DAY(SYSDATE) "Last", LAST_DAY(SYSDATE) - SYSDATE "Days Left"
FROM DUAL;
```

Output:

```
SYSDATE Last Days Left
```

```
23-OCT-97 31-OCT-97 8
```

Example:

```
SELECT MONTHS_BETWEEN(SYSDATE, hiredate) "Months of Service"
FROM DUAL;
```

Example:

```
SELECT ROUND (TO_DATE ('27-OCT-92'),'YEAR')
"New Year" FROM DUAL;
```

Output:

```
New Year
```

```
01-JAN-93
```

Example:

```
SELECT TRUNC(TO_DATE('27-OCT-92','DD-MON-YY'), 'YEAR')
"New Year" FROM DUAL;
```

Output:

```
New Year
```

```
01-JAN-92
```

NUMERIC FUNCTIONS:

The following functions fall under this category.

ABS

ROUND

SIGN

TRUNC

CEIL

SQRT

FLOOR

MOD

Example:

```
SELECT ABS(-25) "Result"
FROM DUAL;
```

Output:

```
Result
```

25

Example:

```
SELECT SIGN(-15) "Result"
FROM DUAL;
```

Output:

Result

-1

Example:

```
SELECT CEIL(25.7) "Result"
FROM DUAL;
```

Output:

Result

26

Example:

```
SELECT FLOOR(25.7) "Result" FROM DUAL;
```

Output:

Result

25

Example:

```
SELECT ROUND(25.29,1) "Round" FROM DUAL;
```

Output:

Round

25.3

```
SELECT ROUND(25.29,-1) "Round" FROM DUAL;
```

Output:

Round

30

Example:

```
SELECT TRUNC(25.29,1) "Truncate" FROM DUAL;
```

Output:

Truncate

25.2

Example:

```
SELECT TRUNC(25.29,-1) "Truncate" FROM DUAL;
```

Output:

Truncate

20

-1 will truncate (make zero) first digit left of the decimal point of 25.29

Example:

```
SELECT MOD(11,4) "Modulus"
FROM DUAL;
```

Output:
Modulus

3

Example:

```
SELECT SQRT(25) "Square root"
FROM DUAL;
```

Output:
Square root

5

CONVERSION FUNCTIONS:

The following functions fall under this category

TO_CHAR
TO_DATE
TO_NUMBER

Example:

```
SELECT TO_CHAR(HIREDATE, 'Month DD, YYYY') "Result"
FROM emp
WHERE ename = 'BLAKE';
```

Output:
Result

May 01, 1981

Example:

```
SELECT TO_CHAR(-10000, 'L99G999D99MI') "Result"
FROM DUAL;
```

Output:
Result

\$10,000.00-

Example:

```
SELECT TO_DATE('January 15, 1989, 11:00 A.M.',
'Month dd, YYYY, HH:MI A.M.',
'NLS_DATE_LANGUAGE = American')
FROM DUAL;
```

Example:

```
SELECT TO_NUMBER('$10,000.00-', 'L99G999D99MI') "Result"
FROM DUAL;
```

Output:

Result

-1000

OTHER SINGLE ROW FUNCTIONS:

The following functions fall under this category.

NVL
VSIZE

Example:

```
SELECT ename, NVL(TO_CHAR(COMM), 'NOT APPLICABLE') "COMMISSION" FROM emp
WHERE deptno = 30;
```

Output:

ENAME COMMISSION

ALLEN 300
WARD 500
MARTIN 1400
BLAKE NOT APPLICABLE
TURNER 0
JAMES NOT APPLICABLE

Example:

```
SELECT ename, VSIZE (ename) "BYTES"
FROM emp
WHERE deptno = 10;
```

Output:

ENAME BYTES

CLARK 5
KING 4
MILLER 6

GROUP FUNCTIONS:

A group function as the name says, gets implemented on more than one record within a column. They can be better understood by looking at their real world examples. There are five very important group functions.

AVG
COUNT
MAX
MIN
SUM

Example:

```
SELECT AVG(sal) "Average" FROM emp;
```

Output:
Average

2077.21429

Example:

```
SELECT COUNT(*) "Total" FROM emp;
```

Output:
Total

18
Total number of records will be returned with a table.

Example:

```
SELECT MAX(sal) "Maximum" FROM emp;
```

Output:
Maximum

5000
On the same line we can find out minimum value using the MIN group function.

Example:

```
SELECT SUM(sal) "Total" FROM emp;
```

Output:
Total

29081

9. Explain in detail about Basic Structure of SQL.(11 marks) (NOV 2017) (DEC2018)

SQL keywords : SQL keywords fall into several groups.

(i). Data retrieval: The most frequently used operation in transactional databases is the data retrieval operation. When restricted to data retrieval commands, SQL acts as a declarative language.

- **SELECT** is used to retrieve zero or more rows from one or more tables in a database. In most applications, SELECT is the most commonly used Data Manipulation Language command. In specifying a SELECT query, the user specifies a description of the desired

result set, but they do *not* specify what physical operations must be executed to produce that result set. Translating the query into an efficient query plan is left to the database system, more specifically to the query optimizer.

- Commonly available keywords related to SELECT include:
 - FROM is used to indicate from which tables the data is to be taken, as well as how the tables JOIN to each other.
 - WHERE is used to identify which rows to be retrieved, or applied to GROUP BY. WHERE is evaluated before the GROUP BY.
 - GROUP BY is used to combine rows with related values into elements of a smaller set of rows.
 - HAVING is used to identify which of the "combined rows" (combined rows are produced when the query has a GROUP BY keyword or when the SELECT part contains aggregates), are to be retrieved. HAVING acts much like a WHERE, but it operates on the results of the GROUP BY and hence can use aggregate functions.
 - ORDER BY is used to identify which columns are used to sort the resulting data.

Example 1:

```
SELECT * FROM books WHERE price > 100.00 and price < 150.00 ORDER BY title
```

This is an example that could be used to get a list of expensive books. It retrieves the records from the *books* table that have a *price* field which is greater than 100.00. The result is sorted alphabetically by book title. The asterisk (*) means to show all columns of the *books* table. Alternatively, specific columns could be named.

Example 2:

```
SELECT books.title, count(*) AS Authors FROM books JOIN book_authors ON  
books.book_number = book_authors.book_number GROUP BY books.title
```

Example 2 shows both the use of multiple tables in a join, and aggregation (grouping). This example shows how many authors there are per book. Example output may resemble:

Title	Authors
-----	-----
SQL Examples and Guide	3
The Joy of SQL	1
How to use Wikipedia	2
Pitfalls of SQL	1

Data manipulation: First there are the standard Data Manipulation Language (DML) elements. DML is the subset of the language used to add, update and delete data.

- INSERT is used to add zero or more rows (formally tuples) to an existing table.
- UPDATE is used to modify the values of a set of existing table rows.
- MERGE is used to combine the data of multiple tables. It is something of a combination of the INSERT and UPDATE elements. It is defined in the SQL:2003 standard; prior to that, some databases provided similar functionality via different syntax, sometimes called an "update".
- TRUNCATE deletes all data from a table (non-standard, but common SQL command).
- DELETE removes zero or more existing rows from a table.

Example:

```
INSERT INTO my_table (field1, field2, field3) VALUES ('test', 'N', NULL);
```

```
UPDATE my_table SET field1 = 'updated value' WHERE field2 = 'N';
```

```
DELETE FROM my_table WHERE field2 = 'N';
```

Data transaction: Transaction, if available, can be used to wrap around the DML operations.

- BEGIN WORK (or START TRANSACTION, depending on SQL dialect) can be used to mark the start of a database transaction, which either completes completely or not at all.
- COMMIT causes all data changes in a transaction to be made permanent.
- ROLLBACK causes all data changes since the last COMMIT or ROLLBACK to be discarded, so that the state of the data is "rolled back" to the way it was prior to those changes being requested.

COMMIT and ROLLBACK interact with areas such as transaction control and locking. Strictly, both terminate any open transaction and release any locks held on data. In the absence of a BEGIN WORK or similar statement, the semantics of SQL are implementation-dependent.

Example:

```
BEGIN WORK;
```

```
UPDATE inventory SET quantity = quantity - 3 WHERE item = 'pants';
```

```
COMMIT;
```

Data definition: The second group of keywords is the Data Definition Language (DDL). DDL allows the user to define new tables and associated elements. Most commercial SQL databases have proprietary extensions in their DDL, which allow control over nonstandard features of the database system.

The most basic items of DDL are the CREATE and DROP commands.

- CREATE causes an object (a table, for example) to be created within the database.

- DROP causes an existing object within the database to be deleted, usually irretrievably. Some database systems also have an ALTER command, which permits the user to modify an existing object in various ways -- for example, adding a column to an existing table.

Example:

```
CREATE TABLE my_table (  
my_field1 INT,  
my_field2 VARCHAR (50),  
my_field3 DATE NOT NULL,  
PRIMARY KEY (my_field1, my_field2) )
```

All DDL statements are auto commit so while dropping a table need to have close look at its future needs.

Data control

The third group of SQL keywords is the Data Control Language (DCL). DCL handles the authorization aspects of data and permits the user to control who has access to see or manipulate data within the database.

Its two main keywords are:

- GRANT — authorizes one or more users to perform an operation or a set of operations on an object.
- REVOKE — removes or restricts the capability of a user to perform an operation or a set of operations.
- **Example:**

```
GRANT SELECT, UPDATE ON my_table TO some_user, another_user;
```

10. Explain in detail about SET operations in SQL(5 Marks) (NOV 2017)

The set operators in SQL are based on the same principles, except they don't have a complement, and can determine the 'difference' between two sets. Here are the operators which we apply to combine two queries:

- **union** - all elements in both queries are returned;
- **intersect** - elements common to both queries are returned;
- **except** - elements in the first query are returned *excluding* any that were returned by the second query.

These are powerful ways of manipulating information, but take note: you can only apply them if the results of the two queries (that are going to be combined) have the same format - that is, the same number of columns, and identical column types! (Although many SQLs try to be helpful by, for example, coercing one data type into another, an idea which is superficially

helpful and fraught with potential for errors). The general format of such queries is illustrated by:

```
select columns1 from table1 union select columns2 from table2
```

Different strokes:

Different vendor implementations of SQL have abused the SQL-92 standard in different ways. For example, Oracle uses **minus** where SQL-92 uses **except**.

An outer join could be used (with modification for NULLs if these little monstrosities are present) to achieve the same result as **except**. Similarly, an inner join (with select distinct) can do what **intersect** does.

Set operators can be combined (as you would expect when playing around with sets) to achieve results that simply cannot be obtained using a single set operator.

Note that there are some restrictions on using **order by** with set operators - **order by** may only be used once, no matter how big the compound statement, and the select list must contain the columns being used for the sort.

Pseudoset operators

Not content with implementing set operators, SQL database creators have also introduced what are called "pseudoset operators". These operators don't fit conveniently into set theory, because they allow multiple rows (redundancies) which are forbidden in true sets. We use the pseudoset operator **union all** to combine the outputs of two queries (all that is done is that the results of the second query are appended to the results of the first). Union all does exactly what we required from a FULL OUTER JOIN.

11. Write Short Notes on Null Values(5 Marks) (NOV 2017)

SQL allows the use of null values to indicate absence of information about the value of an attribute. We can use the special keyword null in a predicate to test for a null value. Wherever a value can appear in SQL, in particular in place of a column value in some row. The deviation from the relational model arises from the fact that the implementation of this ad hoc concept in SQL involves the use of three-valued logic, under which the comparison of NULL with itself does not yield true but instead yields the third truth value, unknown; similarly the comparison NULL with something other than itself does not yield false but instead yields unknown. It is because of this behavior in comparisons that NULL is described as a mark rather than a value.

The relational model depends on the law of excluded middle under which anything that is not true is false and anything that is not false is true; it also requires every tuple in a relation body to have a value for every attribute of that relation. This particular deviation is disputed by some if only because E.F. Codd himself eventually advocated the use of special marks and a 4-valued logic, but this was based on his observation that there are

two distinct reasons why one might want to use a special mark in place of a value, which led opponents of the use of such logics to discover more distinct reasons and at least as many as 19 have been noted, which would require a 21-valued logic. SQL itself uses NULL for several purposes other than to represent "value unknown". For example, the sum of the empty set is NULL, meaning zero, the average of the empty set is NULL, meaning undefined, and NULL appearing in the result of a LEFT JOIN can mean "no value because there is no matching row in the right-hand operand".

12. Write Short Notes on Nested Sub Queries(6 Marks)

NESTED SUBQUERIES: SQL provides a mechanism for nesting sub queries. A subquery is a select-from where expression that is nested within another query. A common use of sub queries is to perform tests for set membership, make set comparisons, and determine set cardinality.

Subqueries: Wouldn't it be nice if you could perform one query, temporarily store the result(s), and then use this result as part of another query? You can, and the trickery used is called a subquery. The basic idea is that instead of a 'static' condition, you can insert a query as part of a where clause! An example is:

Select * from tablename **where** value > (insert select statement here);

Note that in the above query, the inner **select** statement must return just one value (for example, an average). There are other restrictions - the subquery must be in parenthesis, and it must be on the right side of the conditional operator (here, a greater than sign). You can use such sub queries with =, >, <, >=, <= and <>, but not {to the best of my knowledge?} with **between.. And**.

Multiple select sub queries can be combined (using logical operators) in the same statement, but avoid complex queries if you possibly can!

Subqueries that return multiple values

In the above, we made sure that our subquery only returned a single value. Can you think of a way you might use a subquery that returns a list of values? (Such a thing is possible)! Yes, you need an operator that works on lists. An example of such an operator is **in**, which we've encountered before. The query should look something like:

select*from tablename **where** value **in** (insert select statement here);

The assumption is that the nested **select** statement returns a list. The outer shell of the statement can then use **in** to get cracking on the list, looking for a match of value within the

list! There is a surprisingly long list of operators that resemble **in**, and can be used in a similar fashion. Here it is:

Operator	What it does
not in	There is no match with any value retrieved by the nested select statement.
in	We know how this works. Note that = any is a synonym for in that you'll sometimes encounter!
> any	The value is greater than any value in the list produced by the inner submit statement. This is a clumsy way of saying "Give me the value if it's bigger than the smallest number retrieved"!
>= any < any <= any	Similar to > . Usage should be obvious.
> all	Compare this with > any - it should be clear that the condition will only succeed if the value is bigger than the largest value in the list returned by the inner select statement!
>= all < all <= all	If you understand > all , these should present no problem!
= all	You're not likely to use this one much. It implies that (to succeed) all the values returned by the inner subquery are equal to one another and the value being tested!

(It is even possible in some SQL implementations to retrieve a 'table' (multiple columns) using the inner **select** statement, and then use **in** to simultaneously compare multiple values with the rows of the table produced by this inner select. You'll probably never need to use something like this. Several other tricks are possible, including the creation of virtual views by using a subquery

Correlated Subqueries

Whew, we're nearly finished with the sub queries, but there is one more distinct flavour The correlated subquery is a nested **select** statement that can (using trickery) refer to the outer select statement containing it. By so doing, we can successively apply the inner select statement to each line generated by the outer statement! The trick that we use is to create an

alias for the outer select statement, and then refer to this alias in the inner select statement, thus constraining the inner select to dealing with the relevant row. For an example, we return to our tedious drug table:

DrugDosing		
Dosing	DoseMg	Frequency
D1	30	OD
D2	10	OD
D3	200	TDS

Let's say we wanted (for some obscure reason) all doses that are greater than average dose, for each dosing frequency. [Meaningless, but it serves the purposes of illustration].

Select Dose Mg, Frequency **from** DrugDosing fred **where** DoseMg>(selectavg(DoseMg) **from** DrugDosing **where** Frequency = fred. Frequency) ;

The sense of this statement should be clear - we use the outer **select** to choose a row (into which we put DoseMg, and Frequency). We then check whether this row is a candidate (or not) using the **where** statement. What does the **where** statement check? Well, it makes sure that DoseMg is greater than a magic number. The magic number is the average dose for a particular Frequency, the frequency associated with the current row. The only real trickery is how we use the label fred to refer to the current line from within the inner select statement. This label fred is called an alias. We'll learn a lot more about aliases later (Some will argue that aliases are so important you should have encountered them long before, but we disagree). Correlated sub queries are not the only way of doing the above. Using a temporary view is often more efficient, but it's worthwhile knowing both techniques.

13. Write Short Notes on SQL Views(6 Marks)(2 Marks)NOV 2014

View gives a particular user access to selected portions of a table. A view is however more than this - it can limit the ability of a user to update parts of a table, and can even amalgamate rows, or throw in additional columns derived from other columns. Even more complex applications of views allow several tables to be combined into a single view!

How do we make a view?

Interestingly enough, you use a **select** statement to specify the view you wish to create. The syntax is:

Create view *nameofview* **as select** *here have details of select statement*

A variant that you will probably use rather often is:

Create or replace *view name of view* **as select** *here have details of select statement*

(Otherwise you have to explicitly destroy a view - SQL won't simply overwrite a view without the **or replace** instruction, but will instead give you an irritating error).

One *cannot* use an **order by** statement (or something else called a **for update** clause) within a view. There is a whole lot of other convenient things you can do to views. Where you include summary statistics (eg count, sum, etc) in a view it is termed an *aggregate view*. Likewise, using **distinct**, you can have a view on the possible values of a column or grouping of columns.

You can even create a view that is derived from several tables (A *multi-table view*). This is extremely sneaky, as you can largely avoid complex join statements in code which pulls data out of several tables.

How do we limit access to a view?

This is implicit in the way we sneakily use a select statement - to limit access to certain columns, for example, we just **select** the column-names we want access to, and ignore the rest! - if you **insert** a row, then SQL doesn't know what to put into the column entry that's not represented in the view, so it will insert either NULL, or the default value for that column. (Likewise, with **delete**, the entire row will be deleted, even the column entry that is invisible). It is also obvious how we limit access to certain *rows* - we use a **where** clause that only includes the rows we want in the view. Note that (depending on your selection criterion) it is possible to insert a row into a view (and thus the underlying database) and then not be able to see this row in the view! With (in)appropriate selection criteria for the view, one can also alter the properties of rows visible in the view so that they now become hidden!

More draconic constraints are possible. The option **with read only** prevents any modifications to the view (or the underlying database); while **with check option** prevents you from creating rows in the view that cannot be selected (seen) in the view itself. If you use the "with check option", then you should follow this with a name, otherwise SQL will create an arbitrary and quite meaningless name for the constraint that will only confuse you when an error occurs!

How do we remove a view?

More things we can do with views

One trick is to create a "virtual view" and then use this as a "table" which you can update. Instead of specifying a table name, you specify (in parenthesis) the select statement that defines the "virtual view", and all actions are performed on this temporary table! This is particularly useful where you don't have the system authority to create a view, yet need the power of a view.

14. Explain in detail Unified Modeling Language(5 Marks)APRIL 2014.(2 Marks)NOV 2014

UML is officially defined at the Object Management Group (OMG) by the UML metamodel, a Meta-Object Facility metamodel (MOF). Like other MOF-based specifications, the UML metamodel and UML models may be serialized in XML. UML was designed to specify, visualize, construct, and document software-intensive systems.

UML is not restricted to modeling software. UML is also used for business process modeling, systems engineering modeling, and representing organizational structures. The Systems Modeling Language (SysML) is a Domain-Specific Modeling language for systems engineering that is defined as a UML 2.0 profile.

UML has been a catalyst for the evolution of model-driven technologies, which include model-driven development (MDD), model-driven engineering (MDE), and model-driven architecture (MDA). By establishing an industry consensus on a graphic notation to represent common concepts like classes, components, generalization, aggregation, and behaviors, UML has allowed software developers to concentrate more on design and architecture.

UML models may be automatically transformed to other representations (e.g. Java) by means of QVT-like transformation languages, supported by the OMG.

UML is extensible, offering the following mechanisms for customization: profiles and stereotype. The semantics of *extension by profiles* have been improved with the UML 2.0 major revision.

15.Explain in detail about the modification of the database(6 Marks)

Modification of the Database: The content of the database may be modified using the following operations:

- Deletion
- Insertion
- Updating

All these operations are expressed using the assignment operator.

Deletion A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database. Can delete only whole tuples; cannot delete values on only particular attributes. A deletion is expressed in relational $r-E$ where r is a relation and E is a relational algebra query.

Delete all account records in branch-name = "Perryridge"

Insertion To insert data into a relation, we either: specify a tuple to be inserted or write a query whose result is a set of tuples to be inserted s in relational algebra.

Updating A mechanism to change a value in a tuple without changing all values in the tuple s . Use the generalized projection $F_1, F_2, \dots, F_i, (r)$ s . Each F_i is either the i th operator to do this task r attribute of r , if the i th attribute is not updated, or, if the attribute is to be updated F_i is

an expression, involving only constants and the attributes of *r*, which gives the new value for the attribute

Views

In some cases, it is not desirable for all users to see the entire logical model (i.e., all the actual relations stored in the database.) Consider a person who needs to know a customer's loan number but has no need to see the loan amount. This person should see a relation customer-name, loan-number (borrower described, in the relational algebra, by *loan*). Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a view.

View Definition s A view is defined using the create view statement which has the form create view *v* as <query expression where <query expression> is any legal relational algebra query expression. The view name is represented by *v*. Once a view is defined, the view name can be used to refer to the virtual relation that the view generates. View definition is not the same as creating a new relation by evaluating the query expression Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

16.Explain in detail Embedded SQL and Dynamic SQL (APRIL 2014,NOV2014)(11 Marks)

Embedded SQL is a method of combining the computing power of a programming language and the database manipulation capabilities of SQL. It allows programmers to embed SQL statements in programs written in C/C++, Fortran, COBOL, Pascal, etc.

Embedded SQL statements are SQL statements written within application programming languages and preprocessed by a SQL preprocessor before the application program is compiled. There are two types of embedded SQL: static and dynamic.

(2 MARKS – NOV 2010) What is host language?

The SQL standard defines embedding of SQL as *embedded SQL* and the language in which SQL queries are embedded is referred to as the *host language*. A popular host language is C and the mixed C and embedded SQL is called Pro*C in Oracle and Sybase database management systems. Other embedded SQL precompilers are Pro*COBOL, Pro*FORTRAN, Pro*PL/I, Pro*Pascal, and SQL*Module (for Ada).

Using sqlca

- sta-select.sqc (a sample static embedded SQL program)
- dyn-select.sqc (a sample dynamic embedded SQL program)
- makefile (a sample makefile for these two programs)

Using sqlstate (for questions on how to use sqlstate, contact drosu@cs.toronto.edu)

- sta-select.sqc (a sample static embedded SQL program)
- dyn-select.sqc (a sample dynamic embedded SQL program)
- makefile (a sample makefile for these two programs)

Note: before you compile these programs, you need to create the following table and insert some tuples in it.

```
create table video(video_id int not null, \
video_title varchar(30), \
director varchar(20), \
primary key(video_id))
insert into video values(100,'Titanic','John')
insert into video values(200,'Mask','Mary')
insert into video values(300,'Mary had a little lamb','Jim')
```

Dynamic SQL

What Is Dynamic SQL?

Most database applications do a specific job. For example, a simple program might prompt the user for an employee number, then update rows in the EMP and DEPT tables. In this case, you know the makeup of the UPDATE statement at precompile time. That is, you know which tables might be changed, the constraints defined for each table and column, which columns might be updated, and the datatype of each column.

However, some applications must accept (or build) and process a variety of SQL statements at run time. For example, a general-purpose report writer must build different SELECT statements for the various reports it generates. In this case, the statement's makeup is unknown until run time. Such statements can, and probably will, change from execution to execution. They are aptly called *dynamic* SQL statements.

Unlike static SQL statements, dynamic SQL statements are not embedded in your source program. Instead, they are stored in character strings input to or built by the program at run time. They can be entered interactively or read from a file.

Advantages and Disadvantages of Dynamic SQL

Host programs that accept and process dynamically defined SQL statements are more versatile than plain embedded SQL programs. Dynamic SQL statements can be built interactively with input from users having little or no knowledge of SQL.

For example, your program might simply prompt users for a search condition to be used in the WHERE clause of a SELECT, UPDATE, or DELETE statement. A more complex program might allow users to choose from menus listing SQL operations, table and view names, column names, and so on. Thus, dynamic SQL lets you write highly flexible applications.

However, some dynamic queries require complex coding, the use of special data structures, and more runtime processing. While you might not notice the added processing time, you might find the coding difficult unless you fully understand dynamic SQL concepts and methods.

When to Use Dynamic SQL

In practice, static SQL will meet nearly all your programming needs. Use dynamic SQL only if you need its open-ended flexibility. Its use is suggested when one of the following items is unknown at precompile time:

- text of the SQL statement (commands, clauses, and so on)
- the number of host variables
- the datatypes of host variables
- references to database objects such as columns, indexes, sequences, tables, usernames, and views

Requirements for Dynamic SQL Statements

To represent a dynamic SQL statement, a character string must contain the text of a valid SQL statement, but *not* contain the EXEC SQL clause, or the statement terminator, or any of the following embedded SQL commands:

- CLOSE
- DECLARE
- DESCRIBE
- EXECUTE
- FETCH
- INCLUDE
- OPEN
- PREPARE
- WHENEVER

In most cases, the character string can contain *dummy* host variables. They hold places in the SQL statement for actual host variables. Because dummy host variables are just placeholders, you do not declare them and can name them anything you like. For example, Oracle makes no distinction between the following two strings:

```
'DELETE FROM EMP WHERE MGR = :mgr_number AND JOB = :job_title'
```

```
'DELETE FROM EMP WHERE MGR = :m AND JOB = :j'
```

How Dynamic SQL Statements Are Processed

Typically, an application program prompts the user for the text of a SQL statement and the values of host variables used in the statement. Then Oracle *parses* the SQL statement. That is, Oracle examines the SQL statement to make sure it follows syntax rules and refers to valid database objects. Parsing also involves checking database access rights, reserving needed resources, and finding the optimal access path.

Next, Oracle *binds* the host variables to the SQL statement. That is, Oracle gets the addresses of the host variables so that it can read or write their values.

Then Oracle *executes* the SQL statement. That is, Oracle does what the SQL statement requested, such as deleting rows from a table.

The SQL statement can be executed repeatedly using new values for the host variables.

Methods for Using Dynamic SQL

This section introduces four methods you can use to define dynamic SQL statements. It briefly describes the capabilities and limitations of each method, then offers guidelines for choosing the right method. Later sections show you how to use the methods, and include sample programs that you can study.

The four methods are increasingly general. That is, Method 2 encompasses Method 1, Method 3 encompasses Methods 1 and 2, and so on. However, each method is most useful for handling a certain kind of SQL statement, as the following table shows:

Method	Kind of SQL Statement
1	non query without host variables
2	non query with known number of input host variables
3	query with known number of select-list items and input host variables
4	query with unknown number of select-list items or input host variables

Note: The term *select-list item* includes column names and expressions such as SAL * 1.10 and MAX(SAL).

Method 1

This method lets your program accept or build a dynamic SQL statement, then immediately execute it using the EXECUTE IMMEDIATE command. The SQL statement must not be a query (SELECT statement) and must not contain any placeholders for input host variables. For example, the following host strings qualify:

```
'DELETE FROM EMP WHERE DEPTNO = 20'
```

```
'GRANT SELECT ON EMP TO scott'
```

With Method 1, the SQL statement is parsed every time it is executed.

Method 2

This method lets your program accept or build a dynamic SQL statement, then process it using the PREPARE and EXECUTE commands. The SQL statement must not be a query. The number of placeholders for input host variables and the datatypes of the input host variables must be known at precompile time. For example, the following host strings fall into this category:

```
'INSERT INTO EMP (ENAME, JOB) VALUES (:emp_name, :job_title)'
```

```
'DELETE FROM EMP WHERE EMPNO = :emp_number'
```

With Method 2, the SQL statement is parsed just once, but can be executed many times with different values for the host variables. SQL data definition statements such as CREATE and GRANT are executed when they are PREPARED.

18. Elaborate the concept of Advanced SQL NOV 2014.

Built-in Data Types in SQL

- **date:** Dates, containing a (4 digit) year, month and date

Example: **date**'2005-7-27'

- **time:** Time of day, in hours, minutes and seconds.

Example: **time**'09:00:30'**time**'09:00:30.75'

- **timestamp:** date plus time of day

Example: **timestamp**'2005-7-27 09:00:30.75'

- **interval:** period of time

Example: interval '1'day

Subtracting a date/time/timestamp value from another gives an interval value

Interval values can be added to date/time/timestamp values

Can extract values of individual fields from date/time/timestamp

Example: **extract(year from r.starttime)**

Can cast string types to date/time/timestamp

Example: **cast**<string-valued-expression> **as date**

Example: **cast**<string-valued-expression> **as time**

create domain Dollars numeric(12, 2)

create domain Pounds numeric(12,2)

- We cannot assign or compare a value of type Dollars to a value of type Pounds.
- However, we can convert type as below (**cast r.A as Pounds**)

(Should also multiply by the dollar-to-pound conversion-rate)

Large Large-Object Types

- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*:
- **blob:** binary large object --object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
- **clob:** character large object --object is a large collection of character data

When a query returns a large object, a pointer is returned.

Constraints on a Single Relation

- **not null**
- **primary key**
- **unique**

- **check** (*P*),where *P* is a predicate
- **Not Null Constraint**

The Unique Constraint

- **unique**(*A1, A2, ..., Am*)

The unique specification states that the attributes *A1, A2, ...Am* form a candidate key. Candidate keys are permitted to be non null (in contrast to primary keys).

The check clause

- **check** (*P*),where *P* is a predicate

Example: Declare *branch_name* as the primary key for *branch* and ensure that the values of *assets* are non-negative. **create table** *branch*(*branch_name* **char**(15),*branch_city*

char(30),*assets***integer**,**primary key** (*branch_name*),**check**(*assets* >= 0))

- The **check** clause in SQL-92 permits domains to be restricted:
- Use **check** clause to ensure that an *hourly_wage* domain allows only values greater than a specified value.

Referential Integrity

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
- Example: If "Perryridge" is a branch name appearing in one of the tuples in the *account* relation, then there exists a tuple in the *branch* relation for branch "Perryridge".
- Primary and candidate keys and foreign keys can be specified as part of the SQL **create table** statement:
- The primary key clause lists attributes that comprise the primary key.
- The unique key clause lists attributes that comprise a candidate key.
- The foreign key clause lists the attributes that comprise the foreign key and the name of the relation referenced by the foreign key. By default, a foreign key references the primary key attributes of the referenced table.

Referential Integrity in SQL – Example

create table *customer*

(*customer_name***char**(20),*customer_street***char**(30),*customer_city***char**(30),

primary key(*customer_name*))

create table *branch*(*branch_name***char**(15),*branch_city***char**(30),*assets***numeric**(12,2),

primary key(*branch_name*))

create table *account*

(*account_number***char**(10),*branch_name***char**(15),*balance***integer**,**primary key**

(*account_number*),**foreign key**(*branch_name*)**references** *branch*)

create table *depositor*(*customer_name***char**(20),*account_number***char**(10),**primary key**(*customer_name*, *account_number*),**foreign key**(*account_number*)**references** *account*, **foreign key**(*customer_name*)**references** *customer*)

Assertions

- An **assertion** is a predicate expressing a condition that we wish the database always to satisfy.
- An assertion in SQL takes the form

create assertion <assertion-name> **check**<predicate>

- When an assertion is made, the system tests it for validity, and tests it again on every update that may violate the assertion
- This testing may introduce a significant amount of overhead; hence assertions should be used with great care.
- Asserting for all X , $P(X)$ is achieved in a round-about fashion using not exists X such that not $P(X)$

Assertion Example

- Every loan has at least one borrower who maintains an account with a minimum balance or \$1000.00

create assertion *balance_constraint* **check**(**not exists** (**select * from** *loan* **where not exists** (**select * from** *borrower*, *depositor*, *account* **where** *loan.loan_number* = *borrower.loan_number* **and** *borrower.customer_name* = *depositor.customer_name* **and** *depositor.account_number* = *account.account_number* **and** *account.balance* >= 1000)))

Assertion Example

- The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch.

create assertion *sum_constraint* **check**(**not exists** (**select * from** *branch* **where** (**select sum**(*amount*) **from** *loan* **where** *loan.branch_name* = *branch.branch_name*) >= (**select sum** (*amount*) **from** *account* **where** *loan.branch_name* = *branch.branch_name*)))

Authorization

Forms of authorization on parts of the database:

- **Read** -allows reading, but not modification of data.
- **Insert** -allows insertion of new data, but not modification of existing data.
- **Update** -allows modification, but not deletion of data.
- **Delete** -allows deletion of data.

Forms of authorization to modify the database schema (covered in Chapter 8):

- **Index** -allows creation and deletion of indices.
- **Resources** -allows creation of new relations.
- **Alteration** -allows addition or deletion of attributes in a relation.
- **Drop** -allows deletion of relations.

Authorization Specification in SQL

- The **grant** statement is used to confer authorization

grant<privilege list>**on** <relation name or view name> **to**<user list>

<user list> is:a user-id

- **public**, which allows all valid users the privilege granted

A role Granting a privilege on a view does not imply granting any privileges on the underlying relations.

The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

Privileges in SQL

- **select**:allows read access to relation,or the ability to query using the view

Example: grant users *U1*, *U2*, and *U3***select**authorization on the *branch* relation:

grant select on branch to U1, U2, U3

- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

Revoking Authorization in SQL

The **revoke** statement is used to revoke authorization.

revoke <privilege list>**on** <relation name or view name> **from** <user list>

Example:

revoke select on branch from U1, U2, U3<privilege-list> may be **all** to revoke all privileges the revoke may hold.

If <revokee-list> includes **public**, all users lose the privilege except those granted it explicitly.

If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation.

All privileges that depend on the privilege being revoked are also revoked.

19. Table: EMP Emp(empno, ename, job, mgr, hiredate, salary, comm., deptno) [APRIL 2015]

a. List all the employees who have at least one person reporting to them.

SELECT DISTINCT(A.ENAME) FROM EMP A, EMP B WHERE A.EMPNO = B.MGR; or SELECT ENAME FROM EMP WHERE EMPNO IN (SELECT MGR FROM EMP);

b. List the employee details if and only if more than 10 employees are present in department no 10.

SELECT * FROM EMP WHERE DEPTNO IN (SELECT DEPTNO FROM EMP GROUP BY DEPTNO HAVING COUNT(EMPNO)>10 AND DEPTNO=10);

c. List the name of the employees with their immediate higher authority.

SELECT A.ENAME "EMPLOYEE", B.ENAME "REPORTS TO" FROM EMP A, EMP B WHERE A.MGR=B.EMPNO;

d. List all the employees who do not manage any one.

SELECT * FROM EMP WHERE EMPNO IN (SELECT EMPNO FROM EMP MINUS SELECT MGR FROM EMP);

e. List the employee details whose salary is greater than the lowest salary of an employee belonging to deptno 20.

SELECT * FROM EMP WHERE SAL > (SELECT MIN(SAL) FROM EMP GROUP BY DEPTNO HAVING DEPTNO=20);

20. Consider the following relational schema: [April 2017]

Employee (empno, name, office, age)

Books (ISBN, title, authors, publisher)

Loan (empno, isbn, date)

Write the following queries in SQL

(a) Find the name of all employees who have borrowed a book published by McGraw-Hill.

(b) For each publisher, find the name of employees who have borrowed more than five books of that publisher

ANS:

a. select name from employee e, books b, loan l where e.empno = l.empno and l.isbn = b.isbn and b.publisher = 'McGrawHill'

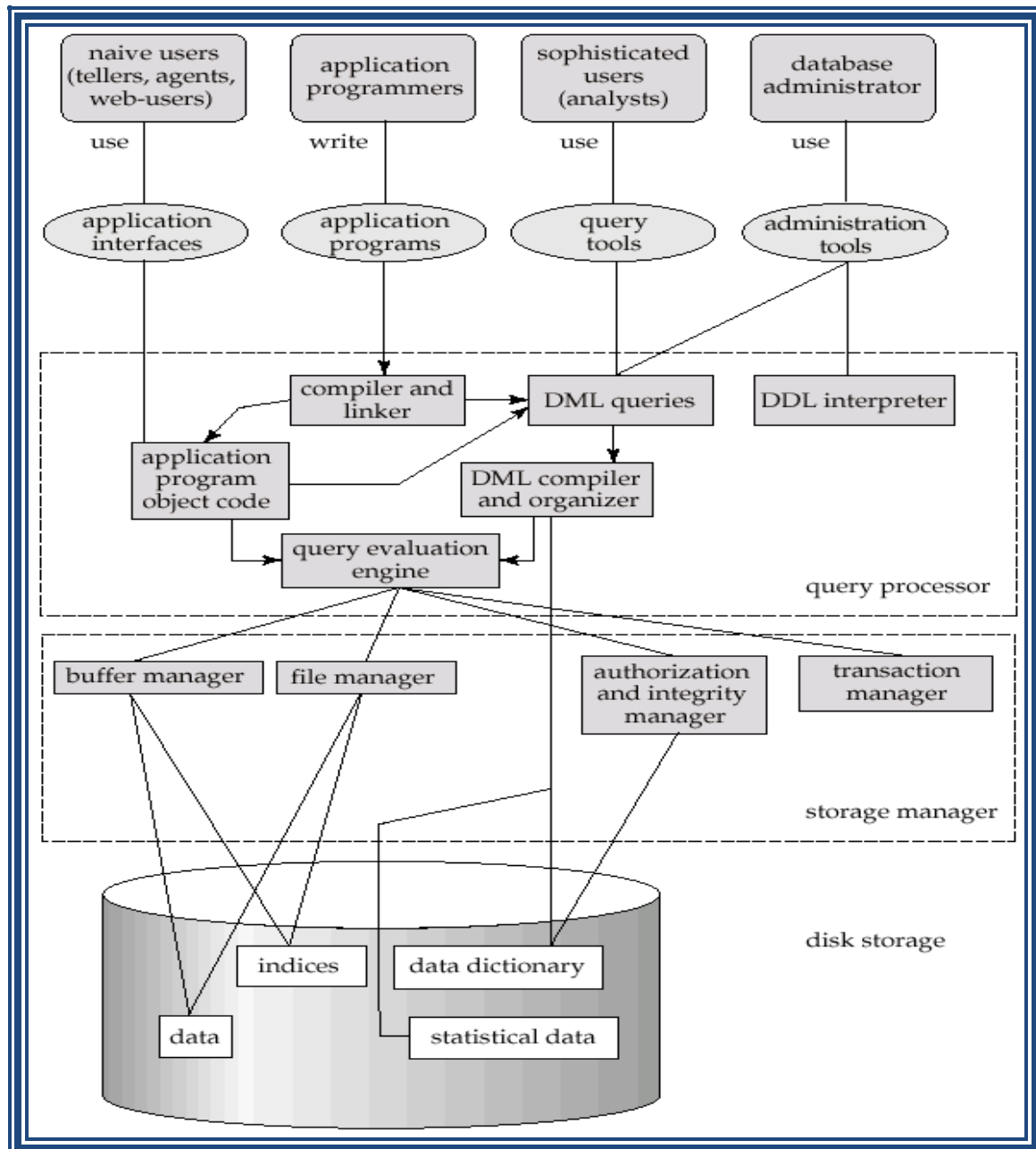
b. select name from employee, loan, books where employee.empno=loan.empno and books.isbn=loan.isbn group by employee.empno, name, books.publisher having count(loan.isbn) >=5

21. List the three design goals for relational databases, and explain why each one of these design goals is desirable. [April 2017]

Lossless join decomposition, dependency preserving decomposition and minimization of repetition of information. This use the smallest use of space, correctness and accuracy, so we use these design goals is desirable.

22. Explain System Structure with neat diagram.

What are the components of database? Explain with neat diagram. (April 2015)
(NOV 2016)(NOV 2017) (APRIL 2018)



Query Processor Components :

- **DML Pre-compiler :** It translates DML statements in a query language into low level instructions that query evaluation engine understands. It also attempts to transform user's request into an equivalent but more efficient form.

- **Embedded DML Pre-compiler** : It converts DML statements embedded in an application program to normal procedure calls in the host language. The Pre-compiler must interact with the DML compiler to generate the appropriate code.
- **DDL Interpreter** : It interprets the DDL statements and records them in a set of tables containing meta data or data dictionary.
- **Query Evaluation Engine** : It executes low-level instructions generated by the DML compiler.

2. Storage Manager Components :

They provide the interface between the low-level data stored in the database and application programs and queries submitted to the system.

- **Authorization and Integrity Manager** : It tests for the satisfaction of integrity constraints checks the authority of users to access data.
- **Transaction Manager** : It ensures that the database remains in a consistent state despite the system failures and that concurrent transaction execution proceeds without conflicting.
- **File Manager** : It manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager** : It is responsible for fetching data from disk storage into main memory and deciding what data to cache in memory.

3. Data Structures :

Following data structures are required as a part of the physical system implementation.

- **Data Files** : It stores the database.
- **Data Dictionary** : It stores meta data (data about data) about the structure of the database.
- **Indices** : Provide fast access to data items that hold particular values.
- **Statistical Data** : It stores statistical information about the data in the database. This information is used by query processor to select efficient ways to execute query.

Database Administrator

The Database Administrator, better known as DBA, is the person (or a group of persons) responsible for the well being of the database management system. S/he has the following functions and responsibilities regarding database management:

1. Definition of the schema, the architecture of the three levels of the data abstraction, data independence.
2. Modification of the defined schema as and when required.
3. Definition of the storage structure i.e. and access method of the data stored i.e. sequential, indexed or direct.
4. Creating new user-id, password etc, and also creating the access permissions that each user can or cannot enjoy. DBA is responsible to create user roles, which are collection of the

permissions (like read, write etc.) granted and restricted for a class of users. S/he can also grant additional permissions to and/or revoke existing permissions from a user if need be.

5. Defining the integrity constraints for the database to ensure that the data entered conform to some rules, thereby increasing the reliability of data.
6. Creating a security mechanism to prevent unauthorized access, accidental or intentional handling of data that can cause security threat.
7. Creating backup and recovery policy. This is essential because in case of a failure the database must be able to revive itself to its complete functionality with no loss of data, as if the failure has never occurred. It is essential to keep regular backup of the data so that if the system fails then all data up to the point of failure will be available from a stable storage. Only those amount of data gathered during the failure would have to be fed to the database to recover it to a healthy status.

23. Construct a database (APRIL 2018)

- branch" table(branch_name, branch_city, assets)
 - "customer" table(customer_name, customer_street, customer_city)
 - "account" table (account_number, branch_name, balance)
 - "loan"table (loan_number, branch_name, amount)
 - "depositor"table (customer_name, account_number)
 - "borrower" table (customer_name, loan_number)
- Give an expression in the relational algebra for each of the following queries.

- a. Find the names of all branches located in “Chicago”.
- b. Find the names of all borrowers who have a loan in branch “Downtown”.

Answer:

- a. $\Pi_{branch_name} (\sigma_{branch_city = \text{“Chicago”}} (branch))$
- b. $\Pi_{customer_name} (\sigma_{branch_name = \text{“Downtown”}} (borrower \bowtie loan))$

UNIVERSITY QUESTIONS

1. Discuss about Data Models(7 marks) **APRIL 2010, (MAY 2019) (Question Number:15)**
2. Discuss about Keys(8 Marks) **APRIL 2010(Question Number:6(iii))**
3. Write about ER Diagrams(8 marks) **APRIL 2010,NOV 2010,APRIL 2014,NOV 2015 (Question Number:15)**
4. Write about UML(7 Marks) **APRIL 2010,APRIL 2014(Question Number:14)**
5. Explain the Structure of relational Databases in detail.(11 marks) **NOV 2010(Question Number:6)**
6. Explain the Aggregate functions in SQL with Examples.(11 marks) **NOV 2010(Question Number:7)**

7. Explain Three basic Notations in E.R Models with example.(11 Marks) APRIL 2011
(Question Number:15)
8. Explain the SQL and its basic structure of SQL QUERY. (11 Marks) APRIL 2011,APRIL 2012,NOV2014, (MAY 2019) (Question Number:8)
9. Explain Relational Algebra (11 Marks) APRIL 2011,APRIL 2012, NOV 2012, NOV 2015(Question Number:7)
10. Discuss in detail about the Aggregate functions in SQL with Examples.(11 marks) NOV 2012(Question Number:7)
- 11.Explain in detail Embedded SQL and Dynamic SQL (APRIL 2014)(11 Marks)
(Question Number:17)
- 12.Discuss about the views in relational model [APR 2014]
(Question Number:6,7)
- 13.Elaborate the concept of Advanced SQL [NOV 2014].
(Question Number: 17, 18)
14. Table: EMP Emp(empno, ename, job, mgr, hiredate, salary, comm., deptno)
 - a.List all the employees who have at least one person reporting to them.
 - b.List the employee details if and only if more than 10 employees are present in department no 10.
 - c. List the name of the employees with their immediate higher authority.
 - d. List all the employees who do not manage any one.
 - e. List the employee details whose salary is greater than the lowest salary of an employee belonging to deptno 20. [APR 2015]. (Question Number:19)
- 15.Discuss with examples various operations of Relation Algebra Operations. [APR 2015].
NOV 2015(Question Number:20)
16. Short note in Data Definition in SQL. [DEC 2016]
17. Discuss about set operation in SQL. [DEC 2016]
18. Consider the following relational schema: [April 2017] (Question Number:21)
Employee (empno, name, office, age)
Books(ISBN, title, authors, publisher)
Loan (empno, isbn, date)
Write the following queries in SQL
 - a. Find the name of all employees who have borrowed a book published by McGraw-Hill.
 - b. For each publisher, find the name of employees who have borrowed more than five books of that publisher
20. List the three design goals for relational databases, and explain why each one of these design goals is desirable. [April 2017] (Question Number:22)

21. **What are the components of database? Explain with neat diagram. (NOV 2017)**

(Question Number:23)

22. **Explain SQL basic operations, set operations and null values. (NOV 2017) (Question Number:9,10,11)**

23. **Construct a database (APRIL 2018)**

- "branch" table(branch_name, branch_city, assets)
- "customer" table(customer_name, customer_street, customer_city)
- "account" table (account_number, branch_name, balance)
- "loan"table (loan_number, branch_name, amount)
- "depositor"table (customer_name, account_number)
- "borrower" table (customer_name, loan_number)

Give an expression in the relational algebra for each of the following queries.

- a. Find the names of all branches located in "Chicago".
- b. Find the names of all borrowers who have a loan in branch "Downtown".